PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING

COURSE: SOFTWARE ARCHITECTURE

HOTEL MANAGEMENT SYSTEM

INSTRUCTOR: Vũ Quang Dũng

GROUP 7: Nguyễn Hưng Thành (23010688 – K17 KTPM)

CREDIT CLASS: CSE703110-1-2-25(N01)

Ha Noi, March 6, 2026

Contents

1. Executive Summary

This project implements a Hotel Booking Management System that supports the end-to-end workflow of a real hotel: customers can browse rooms, create bookings, make payments, manage their wallet, and view booking history; receptionists can check guests in and out and manage service requests; administrators can manage rooms, services, staff, coupons, and reports. The system is architected using a layered architecture on top of Flask, with clear separation between controllers (HTTP layer), services (business logic), repositories (data access), and models (domain entities).

The backend lives under SRC/Arch and exposes REST-style JSON APIs (e.g. /api/bookings, /api/rooms, /api/auth/login, /api/wallet) through controller classes such as booking_controller.py, room_controller.py, user_controller.py, payment_controller.py, coupon_controller.py, checkin_controller.py, staff_controller.py, report_controller.py, and wallet_controller.py. These controllers rely on service classes (e.g. booking_service.py, room_service.py, payment_service.py, coupon_service.py) that encapsulate business logic and call repository classes (e.g. booking_repository.py, room_repository.py, coupon_repository.py) to access a MySQL database.

The frontend is implemented in SRC/UI as a set of HTML/CSS/JS pages for different roles: public pages for browsing rooms, authentication pages for login and registration, user pages for dashboard, my bookings, payment, and wallet, and admin pages for room and staff management and reports. The final outcome is a working multi-role hotel management application where the architecture supports modifiability, maintainability, and basic security and scalability for small-to-medium usage.

2. Project Requirements & Goals

2.1 Use Case Modeling

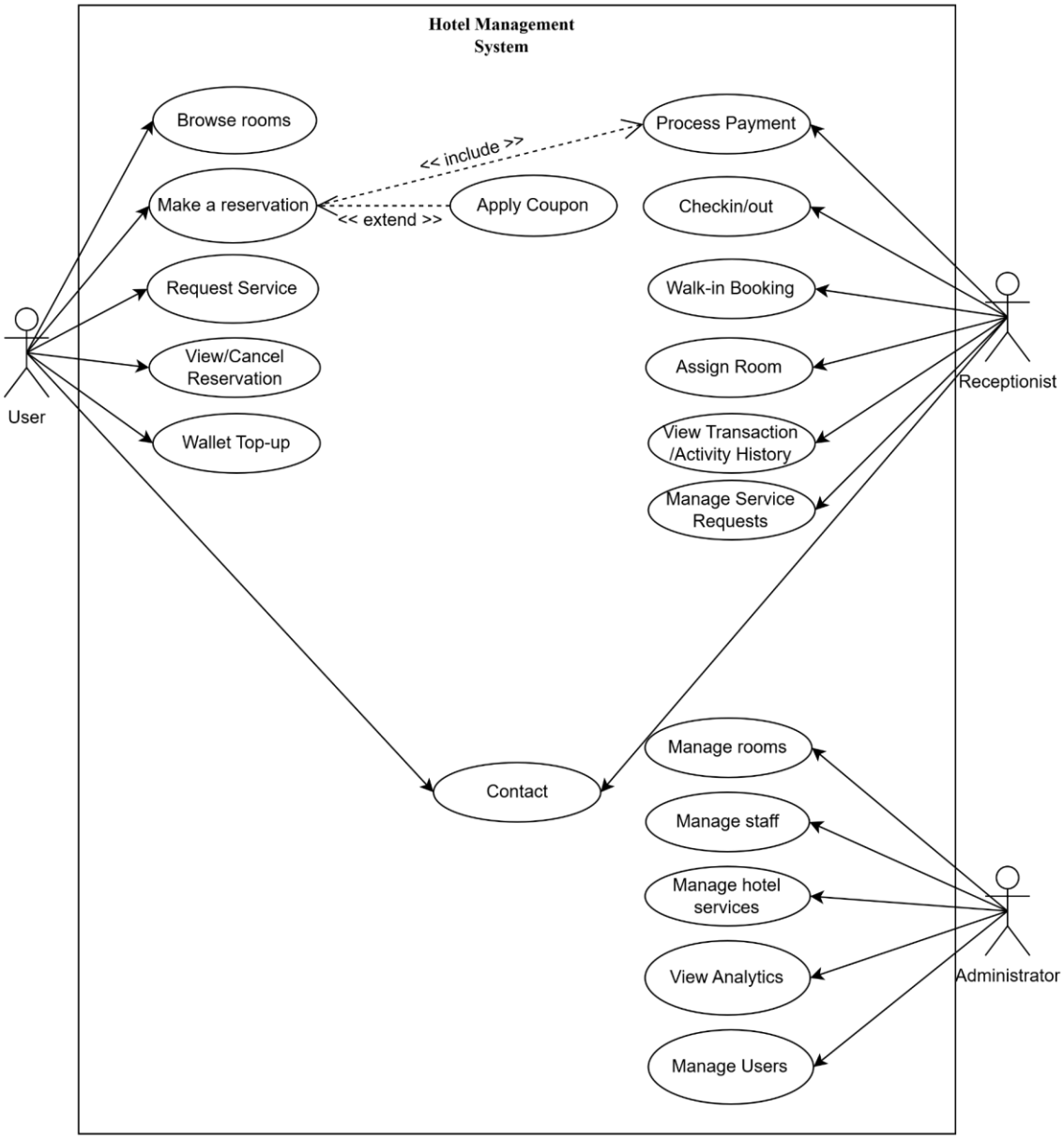

Figure 1: Use case diagram

The use case diagram describes the functional scope of the hotel management system through three main user groups and their use cases, all inside the system boundary labelled “Hotel Management System”.

Roles and functional groups

- User – left: self-service actions: browse rooms, make a reservation, request service, view/cancel reservation, and wallet top-up. Making a reservation always includes payment (Process Payment – include) and can optionally apply a discount (Apply Coupon – extend).
- Receptionist – top right: daily operations: direct payment (Process Payment), check-in/check-out (Checkin/out), walk-in booking (Walk-in Booking), assign room (Assign

Room), view transaction and activity history (View Transaction/Activity History), and manage service requests (Manage Service Requests).

- Administrator – bottom right: configuration and master data: manage rooms (Manage rooms), manage staff (Manage staff), manage hotel services (Manage hotel services), view analytics (View Analytics), and manage users (Manage Users).

Overall, the diagram clarifies functional scope per role, the include/extend relationships around reservation, and the separation between self-service (guest), operations (receptionist), and configuration (administrator), making it suitable for system architecture analysis and design.

2.1.1 Interface Specification and Development UC1 – Users.

UC-01: Login

ID and Name:	UC-01: Login
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User (any role: user / receptionist / administrator)
Description:	This use case allows a user to authenticate with username and password to access protected features such as booking, payment, wallet, and administrative functions (depending on role).
Trigger:	User selects “Login” and submits credentials.
Preconditions:	- The user has already registered (exists in the users table). - The backend service is running and database connection is available.
Postconditions:	1. Success - The system returns user information (including user_id and role) to the client. - The client stores the returned user_id and includes it in subsequent protected API calls via HTTP header: X-User-Id: <user_id>. 2. Failure - No authenticated context is established; protected resources remain inaccessible.
Normal flow:	1. User opens the login page in the UI. 2. User enters username and password. 3. User clicks “Login”. 4. Client sends POST /api/auth/login with { "username": "...", "password": "..." }. 5. System validates request format. 6. System authenticates users against MySQL credentials. 7. System returns user_id and role.

	8. Client stores user_id and uses X-User-Id for protected requests.
Alternative Flows:	A1 – Missing credentials → 400. A2 – Invalid username/password → 401/400 with message "Invalid username or password."
Exceptions:	E1 – Database connection error → 500.
Priority:	High
Frequency of Use:	Very frequent (daily)
Business rules:	- Username and password are required fields. - Only authenticated users can access endpoints protected by @require_auth. - Authorization is role-based (RBAC) via middleware decorators: - Administrator: full access to admin endpoints. - Receptionist: access to receptionist endpoints (bookings list, check-in/out, etc.). - User: access to own bookings, payments, wallet, and service requests.

UC-02: Register

ID and Name:	UC-02: Register
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User (any role: user / receptionist / administrator)
Description:	This use case allows a guest to create a new user account in the system.
Trigger:	Guest submits the registration form.
Preconditions:	None.
Postconditions:	1. Success - A new user record is created in the users table. 2. Failure - No user record is created.
Normal flow:	1. Guest opens the Register page. 2. Guest enters username, email, password, full_name, phone. 3. Client sends POST /api/users with JSON body. 4. System validates input data.

	5. System saves the new user to MySQL. 6. System returns created user information to client.
Alternative Flows:	A1 – Missing/invalid fields 1. Guest submits missing required fields. 2. System returns 400 with validation error.
Exceptions:	E1 – Database error - System returns 500 if database operation fails..
Priority:	High
Frequency of Use:	Medium
Business rules:	- Required fields must not be empty. - Username/email should be unique (if enforced by DB/repository).

UC-03: Browse Rooms (List/Search/Detail)

ID and Name:	UC-03: Browse Rooms (List/Search/Detail)
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows browsing the room catalog: list rooms, search rooms, and view room detail
Trigger:	Actor opens Rooms page or Room Detail page.
Preconditions:	None (browsing rooms is optional_auth).
Postconditions:	Room list/detail is displayed to the actor.
Normal flow:	1. Client requests a room list via GET /api/rooms. 2. (Optional) Client searches via GET /api/rooms/search?room_type=...&status=.... 3. Client views detail via GET /api/rooms/<room_id>. 4. If a room has an image_url, the client loads the image via GET /uploads/<path>.
Alternative Flows:	A1 – Room not found → 404 for GET /api/rooms/<room_id>.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Very frequent

Business rules:	Current search endpoint filters by room_type and status.

UC-04: Create Booking (Reservation)

ID and Name:	UC-04: Create Booking (Reservation)
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows an authenticated user to create a booking for a room type and date range. The system checks availability, reserves a room, and persists the booking.
Trigger:	The user confirms booking details in the UI.
Preconditions:	- User is authenticated (X-User-Id is present). - At least one room of selected type is available for the requested date range.
Postconditions:	- A booking record is created (status pending). - A selected room is reserved (status reserved).
Normal flow:	1. User enters guest_name, room_type, check_in_date (and optional check_out_date). 2. Client sends POST /api/bookings with required JSON and X-User-Id header. 3. System validates input and date rules. 4. System checks availability for the date range. 5. System selects a room and sets room status to reserved. 6. System saves booking to MySQL with status pending and room_id. 7. The system returns the created booking to the client.
Alternative Flows:	A1 – No available rooms → 400 with message. A2 – Invalid date format/past date → 400.
Exceptions:	E1 – Error reserving room or saving booking → 500 (room should be released if needed).
Priority:	High
Frequency of Use:	Frequent
Business rules:	Prevent overlapping bookings by date-range availability check.

UC-05: View My Bookings

ID and Name:	UC-05: View My Bookings
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows the user to view their booking history.
Trigger:	User opens “My Bookings”.
Preconditions:	- User authenticated (X-User-Id).
Postconditions:	- A list of user bookings is returned and displayed.
Normal flow:	1. Client sends GET /api/bookings/my with X-User-Id. 2. System returns bookings for that user.
Alternative Flows:	A1 – User not found in request context → 404.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Frequent
Business rules:	Users can only view their own bookings.

UC-06: Cancel Booking (Refund if applicable)

ID and Name:	UC-06: Cancel Booking (Refund if applicable)
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows a user to cancel a booking. The system releases the reserved room and may refund wallet payments and mark payments as refunded.
Trigger:	User selects “Cancel booking”.
Preconditions:	- User authenticated. - Booking exists and is not checked_in/checked_out/cancelled. - No check-in record exists for this booking.
Postconditions:	1. Success - Booking becomes cancelled. - Reserved room becomes available (if applicable). - Wallet refund may be applied (if paid by wallet). 2. Failure - Booking remains unchanged.
Normal flow:	1. Client sends DELETE /api/bookings/<booking_id> with X-

	User-Id. 2. The system validates booking status and check-in constraints. 3. The system releases reserved rooms if needed. 4. System cancels booking (status cancelled). 5. (Optional) System refunds wallet payment and updates payment status to refunded. 6. The system returns a success message.
Alternative Flows:	A1 – Booking already checked_in/checked_out → 400. A2 – Check-in record exists → 400.
Exceptions:	E1 – Database error → 500.
Priority:	Medium
Frequency of Use:	Medium
Business rules:	Cannot cancel after check-in or if check-in record exists.

UC-07: Wallet Top-up

ID and Name:	UC-07: Wallet Top-up
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows a user to add money to their internal wallet.
Trigger:	User submits a top-up amount.
Preconditions:	- User authenticated.
Postconditions:	1. Success - Wallet balance increases by the amount. 2. Failure - Wallet balance unchanged.
Normal flow:	1. Client sends POST /api/wallet/topup with { "amount": ... } and X-User-Id. 2. System validates amount > 0 and numeric. 3. System updates wallet balance in MySQL. 4. System returns updated wallet.
Alternative Flows:	A1 – amount invalid/<=0 → 400.
Exceptions:	E1 – Database error → 500.
Priority:	Medium

Frequency of Use:	Medium
Business rules:	Amount must be numeric and > 0.

UC-08: View Wallet Balance

ID and Name:	UC-08: View Wallet Balance
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows an authenticated user to view wallet balance. If wallet does not exist, it is auto-created.
Trigger:	User opens Wallet page.
Preconditions:	- User authenticated (X-User-Id).
Postconditions:	1. Success - Wallet object (balance) is returned to the client. 2. Failure - Wallet is not returned.
Normal flow:	1. Client sends GET /api/wallet with X-User-Id. 2. System finds wallet by user_id; if missing, creates wallet with balance 0. 3. System returns wallet object.
Alternative Flows:	A1 – Missing authentication header → 401.
Exceptions:	E1 – Database error → 500.
Priority:	Medium
Frequency of Use:	Medium
Business rules:	- Wallet belongs to the authenticated user only.

UC-09: Process Payment

ID and Name:	UC-09: Process Payment
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows an authenticated user to pay for a booking using supported methods (wallet, cash, bank_transfer, credit_card) and records a payment transaction.
Trigger:	User clicks “Pay” on the payment page for a booking.

Preconditions:	- User authenticated (X-User-Id). - Booking exists and is not cancelled.
Postconditions:	1. Success - A payment record is created (status completed). - If payment_method = wallet, wallet balance is deducted accordingly. 2. Failure - No completed payment is recorded; wallet balance unchanged.
Normal flow:	1. Client sends POST /api/payments with { booking_id, amount, payment_method, optional transaction_id } and X-User-Id. 2. System validates amount > 0, payment_method is allowed, and booking exists. 3. If wallet payment: system checks balance and deducts. 4. System saves payment in MySQL and returns payment result.
Alternative Flows:	A1 – Invalid amount/method → 400. A2 – Wallet insufficient balance → 400. A3 – Booking not found → 404/400.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Frequent
Business rules:	- Payment must be tied to booking_id and user_id (from X-User-Id context).

UC-10: Apply Coupon

ID and Name:	UC-10: Apply Coupon
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case validates a coupon code and returns coupon details so the frontend can compute the discount for booking/payment.
Trigger:	User enters a coupon code and clicks “Apply”.
Preconditions:	- User authenticated (X-User-Id). - Coupon exists in database.
Postconditions:	1. Success - Coupon details are returned (discount_type, discount_value,

	validity). 2. Failure - Coupon is rejected with a validation error.
Normal flow:	1. Client sends POST /api/coupons/apply with { code } and X-User-Id. 2. System loads coupon by code and validates: active status + valid date range. 3. System returns coupon information to the client.
Alternative Flows:	A1 – Coupon code not found → 400. A2 – Coupon inactive/expired/not yet valid → 400.
Exceptions:	E1 – Database error → 500.
Priority:	Medium
Frequency of Use:	Medium
Business rules:	- Coupon must be active and within valid_from/valid_to.

UC-11: Service Request

ID and Name:	UC-11: Service Request
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows an authenticated user to request additional hotel services during a stay (e.g., dining, laundry) for a booking.
Trigger:	User selects a service and submits a request.
Preconditions:	- User authenticated (X-User-Id). - Booking exists. - Service exists and is available.
Postconditions:	1. Success - A service_requests record is created with status pending. 2. Failure - No service request is created.
Normal flow:	1. Client sends POST /api/services/request with { booking_id, service_id, quantity } and X-User-Id. 2. System validates booking and service existence and quantity > 0. 3. System creates the service request in MySQL and returns the created request.

Alternative Flows:	A1 – Invalid quantity or missing fields → 400. A2 – Booking/service not found → 404/400.
Exceptions:	E1 – Database error → 500.
Priority:	Medium
Frequency of Use:	Medium
Business rules:	- Service request is linked to a booking and may be included in checkout total.

UC-12: Contact with Receptionist/Admin

ID and Name:	UC-12: Contact with Receptionist/Admin
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	User
Description:	This use case allows a user to contact the receptionist or administrator (e.g., to request services or ask for more information about rooms) via the Contact/Support pages. Messages are exchanged through the in-app chat UI , conversation state is stored client-side (e.g., localStorage) in the current implementation.
Trigger:	User opens the Contact page and sends a message.
Preconditions:	- User authenticated (X-User-Id).
Postconditions:	1. Success - User's message is stored and visible to staff in the admin-messages view. 2. Failure - Message is not delivered or not visible (e.g., client-side storage full).
Normal flow:	1. User navigates to Contact/Support 2. User types a message in the chat input and submits. 3. Client stores the message and updates the UI. 4. Receptionist/Admin can view and reply.
Alternative Flows:	A1 – User not logged in: some implementations may still allow sending a message with a guest identifier.
Exceptions:	E1 – Client-side storage failure → message may not persist.
Priority:	Medium
Frequency of Use:	Medium

Business rules:	- Contact is implemented as client-side chat; no backend API is required for the current design.

2.1.2 Interface Specification and Development UC2 – Receptionist.

UC-13: View All Bookings

ID and Name:	UC-13: View All Bookings
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows receptionists to view all bookings for daily operations.
Trigger:	Receptionist opens booking management screen.
Preconditions:	- Receptionist authenticated and has receptionist/admin role.
Postconditions:	- System returns the booking list.
Normal flow:	1. Client sends GET /api/bookings with X-User-Id. 2. System authorizes role (receptionist/admin). 3. System returns bookings.
Alternative Flows:	A1 – Access denied (wrong role) → 403.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Frequent
Business rules:	- Only receptionist/admin can access this endpoint.

UC-14: Walk-in Booking

ID and Name:	UC-14: Walk-in Booking
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows receptionists to confirm a booking for walk-in guests at the counter.
Trigger:	Receptionist enters guest information and booking details.
Preconditions:	- Receptionist authenticated. - At least one room is available for the requested dates and room_type.

Postconditions:	1. Success - A booking is created (status pending) and a room is reserved. 2. Failure - Booking is not created.
Normal flow:	1. Receptionist inputs guest_name, room_type, check_in_date, optional check_out_date. 2. Client sends POST /api/bookings with booking JSON and X-User-Id (receptionist). 3. System validates input and checks availability (date-range overlap prevention). 4. System reserves a room and saves booking in MySQL.
Alternative Flows:	A1 – No available rooms → 400. A2 – Invalid dates → 400.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Medium
Business rules:	- Walk-in booking uses the same booking creation logic as online booking.

UC-15: Assign Room to Booking

ID and Name:	UC-15: Assign Room to Booking
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows receptionists to assign a room to a booking.
Trigger:	Receptionist selects a booking and assigns a room.
Preconditions:	- Receptionist authenticated. - Booking exists. - Room exists and is not occupied.
Postconditions:	- Booking is updated with room_id. - Room status may be updated based on rules.
Normal flow:	1. Receptionist chooses booking_id and room_id. 2. Client sends POST /api/rooms/assign with { room_id, booking_id } and X-User-Id. 3. System validates room and booking.

	4. System updates booking and room state. 5. System returns assigned room info.
Alternative Flows:	A1 – Room not found or occupied → 400/404. A2 – Booking not found → 404.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Medium
Business rules:	- Cannot assign an occupied room.

UC-16: Process Check-in

ID and Name:	UC-16: Process Check-in
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows receptionists to check in a guest for a booking and create a check-in record.
Trigger:	Receptionist confirms check-in for a booking.
Preconditions:	- Receptionist authenticated. - Booking exists. - Booking not already checked in; no existing check-in record.
Postconditions:	- Check-in record is created in MySQL. - Booking and room statuses are updated accordingly.
Normal flow:	1. Receptionist inputs booking_id (and optional room_id). 2. Client sends POST /api/checkins with JSON body and X-User-Id. 3. System validates booking and receptionist role. 4. System creates check-in record. 5. System updates booking status to checked_in and room status to occupied. 6. System returns check-in information.
Alternative Flows:	A1 – Existing check-in already exists → 400. A2 – No available room for booking type → 400.
Exceptions:	E1 – Database error → 500.
Priority:	High

Frequency of Use:	Frequent
Business rules:	- A booking can be checked in only once.

UC-17: Process Check-out

ID and Name:	UC-17: Process Check-out
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows receptionists to check out a guest, calculate total charges (room + services), and create a checkout record.
Trigger:	Receptionist confirms checkout for a booking.
Preconditions:	- Receptionist authenticated. - A check-in exists for the booking.
Postconditions:	1. Success - Checkout record is created. - Booking status becomes checked_out and room status becomes available. 2. Failure - No checkout record is created; states unchanged.
Normal flow:	1. Receptionist requests summary via GET /api/checkouts/summary/<booking_id>. 2. System calculates total (booking.total_price + service_requests totals). 3. Receptionist sends POST /api/checkouts with { booking_id, total_amount, receptionist_id, optional checkout_time }. 4. System validates check-in exists and saves checkout record; updates booking/room status.
Alternative Flows:	A1 – No check-in exists → 400.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Frequent
Business rules:	- Cannot check out without a prior check-in.

UC-18: View Transaction & Activity History

ID and Name:	UC-18: View Transaction & Activity History

Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows receptionists to view historical records of bookings, payments, and service requests for operations and auditing.
Trigger:	Receptionist opens history dashboards (bookings, payments, service requests).
Preconditions:	- Receptionist authenticated with receptionist/admin role.
Postconditions:	- Lists are returned for display.
Normal flow:	1. Receptionist views bookings via GET /api/bookings. 2. Receptionist views payments via GET /api/payments. 3. Receptionist views service requests via GET /api/service-requests.
Alternative Flows:	A1 – Access denied → 403.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Frequent
Business rules:	- Only receptionist/admin can access global histories.

UC-19: Manage Service Requests

ID and Name:	UC-19: Manage Service Requests
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows receptionists to view all service requests and update each request status (pending, in_progress, completed, cancelled).
Trigger:	Receptionist selects a service request and updates its status.
Preconditions:	- Receptionist authenticated with receptionist/admin role.
Postconditions:	1. Success - Service request status is updated in MySQL. 2. Failure - Status unchanged.
Normal flow:	1. Receptionist loads all requests via GET /api/service-requests. 2. Receptionist selects a request and sends PUT /api/service-

	requests/<request_id> with { status, optional notes }. 3. System validates status and updates the record; returns updated request.
Alternative Flows:	A1 – Invalid status → 400. A2 – Request not found → 404.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Frequent
Business rules:	- Status must be one of: pending, in_progress, completed, cancelled.

UC-20: Contact with Guest

ID and Name:	UC-20: Contact with Guest
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Receptionist
Description:	This use case allows the receptionist or administrator to view guest messages and reply to them through the contact/support channels. Staff use the admin-messages page to see conversations and send replies; messages are stored client-side (e.g., localStorage) and synced so the guest can see replies when they reopen the Contact page.
Trigger:	Receptionist/Admin opens the Contact/Messages management page (admin-messages.html) and replies to a guest.
Preconditions:	- Receptionist or Admin is authenticated (X-User-Id with role receptionist or administrator). - At least one guest has sent a message (stored in chat data).
Postconditions:	1. Success - Staff reply is stored and associated with the guest conversation; guest can see it when viewing the chat. 2. Failure - Reply is not stored or not visible to guest (e.g., client-side error).
Normal flow:	1. Staff opens admin-messages.html and selects a guest/conversation from the list. 2. Staff views the message history (messages loaded from localStorage).

	3. Staff types a reply and submits; client appends the message to the conversation and updates localStorage. 4. Guest can see the reply when they next open the Contact page or refresh the chat.
Alternative Flows:	A1 – No conversations yet → empty state is shown.
Exceptions:	E1 – Client-side storage or script error → reply may not persist.
Priority:	High
Frequency of Use:	Frequent
Business rules:	- Implemented in SRC/UI; no backend API; data is stored in browser localStorage.

2.1.3 Interface Specification and Development UC3 – Administrator.

UC-21: Manage Rooms (CRUD)

ID and Name:	UC-21: Manage Rooms (CRUD)
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Administrator
Description:	This use case allows administrators to create, update, and delete rooms, including changing room status and managing room capacity and images.
Trigger:	Admin performs room management operations from admin UI.
Preconditions:	Admin authenticated with administrator role.
Postconditions:	1. Success - Room records are created/updated/deleted in MySQL. 2. Failure - No changes are applied.
Normal flow:	1. Admin creates a room via POST /api/rooms. 2. Admin updates room fields via PUT /api/rooms/<room_id>. 3. Admin deletes a room via DELETE /api/rooms/<room_id>.
Alternative Flows:	A1 – Cannot delete room with check-in history → 400. A2 – Invalid room data (price <= 0, capacity <= 0) → 400.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Medium

Business rules:	- Only administrator can create/update/delete rooms. - Room price must be positive; capacity (if provided) must be positive.

UC-22: View Analytics

ID and Name:	UC-22: View Analytics
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Administrator
Description:	This use case allows administrators to view analytics (revenue, occupancy, booking) and store them.
Trigger:	Admin requests a report for a given period.
Preconditions:	- Admin authenticated with administrator role.
Postconditions:	1. Success - A report record is created and stored in reports table. 2. Failure - No report is stored.
Normal flow:	1. Admin requests revenue report via POST /api/reports/revenue (with optional dates). 2. System aggregates payment/booking data. 3. System saves report and returns report object. 4. Admin may repeat for occupancy and booking reports.
Alternative Flows:	A1 – Invalid date range input → 400.
Exceptions:	E1 – Database error during aggregation/save → 500.
Priority:	Medium
Frequency of Use:	Medium
Business rules:	- Only administrator can generate reports.

UC-23: Manage Staff Accounts (CRUD)

ID and Name:	UC-23: Manage Staff Accounts (CRUD)
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Administrator
Description:	This use case allows administrators to create, update, and delete staff records.

Trigger:	Admin performs staff management operations from admin UI.
Preconditions:	- Admin authenticated with administrator role.
Postconditions:	1. Success - Staff records are created/updated/deleted in MySQL. 2. Failure - No changes are applied.
Normal flow:	1. Admin creates staff via POST /api/staff. 2. Admin updates staff via PUT /api/staff/<staff_id>. 3. Admin deletes staff via DELETE /api/staff/<staff_id>.
Alternative Flows:	A1 – Invalid staff data (missing full_name/email, invalid email format) → 400.
Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Medium
Business rules:	- Staff email must be unique.

UC-24: Manage Users Accounts (CRUD)

ID and Name:	UC-24: Manage Users Accounts (CRUD)
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Administrator
Description:	This use case allows administrators to list, update, and delete user accounts.
Trigger:	Admin performs user management from admin UI.
Preconditions:	- Admin authenticated with administrator role.
Postconditions:	1. Success - User records are updated/deleted in MySQL. 2. Failure - No changes are applied.
Normal flow:	1. Admin lists users via GET /api/users. 2. Admin updates a user via PUT /api/users/<user_id>. 3. Admin deletes a user via DELETE /api/users/<user_id>.
Alternative Flows:	A1 – User not found → 404. A2 – Invalid update data (duplicate email/invalid role) → 400.

Exceptions:	E1 – Database error → 500.
Priority:	High
Frequency of Use:	Medium
Business rules:	- Only admin can manage users list and roles.

UC-25: Manage Services (CRUD)

ID and Name:	UC-25: Manage Services (CRUD)
Create by:	Nguyễn Hưng Thành / Date created: 6/2/2026
Primary Actor:	Administrator
Description:	This use case allows administrators to create, update, and delete hotel services (spa, dining, laundry, etc.).
Trigger:	Admin updates the service catalog.
Preconditions:	- Admin authenticated with administrator role.
Postconditions:	1. Success - Service catalog is updated in MySQL. 2. Failure - No changes are applied.
Normal flow:	1. Admin creates a service via POST /api/services. 2. Admin updates a service via PUT /api/services/<service_id>. 3. Admin deletes a service via DELETE /api/services/<service_id>.
Alternative Flows:	A1 – Invalid service data (price <= 0) → 400.
Exceptions:	E1 – Database error → 500.
Priority:	Medium
Frequency of Use:	Medium
Business rules:	- Service price must be positive.

2.1.4 Description of Layers

Layer	Purpose/Responsibility	Output/Artifact
1. Presentation Layer (UI)	Handles HTTP requests, user authentication, session management, and renders the user interface. It ensures the separation of the frontend (HTML/CSS) from the backend logic.	Controllers (e.g., room_controller.py, booking_controller.py,...), Templates (HTML views), Views (JSON), HTML/CSS/JS in SRC/UI.
2. Business Logic Layer (Service)	Contains the core business rules, validation logic, and transaction management. It orchestrates data access and handles integrations with external systems (like the internal payment).	Services (e.g., booking_service.py, payment_service.py, room_service.py,...).
3. Persistence Layer (Data Access)	Responsible for mapping business objects to database entities and executing CRUD (Create, Read, Update, Delete) operations. It abstracts the database queries from the business logic.	Repositories/DAOs (e.g., booking_repository.py, room_repository.py,...).
4. Data Layer	The physical database storage system where all application data is persisted.	Database Schema (MySQL), Tables (users, rooms, bookings, payments, wallets, …).